

CSCI 4210 — Operating Systems

Sample Exam 1 Questions and Prep

Overview

- Exam 1 will be in during our testing block on **Wednesday, February 21, 2024**. For most of you this will be from **6:00-7:50PM** in the **West Hall Auditorium** (please arrive early and sit with empty seats next to you on both sides).
- Those of you with accommodations will instead report to **Darrin Communiation Center (DCC) 235** at **4:00**
- Exam 1 s targeted for **105 minutes**; therefore, if you have 50% additional time, you will have **160 minutes**; if you have 100% additional time, you will have **210** minutes.
- You may bring one double-sided or two single-sided 8.5"x11" crib sheets containing anything you would like; crib sheets will not be collected.
- No calculators, laptops, books, phones, etc.
- Exam 1 will be scanned and graded in Submitty.
- Exam 1 will cover everything we have done thus far this semester through lecture on Friday 2/16.
- Use the sample questions and suggested topics below to prepare. Also study the quizzes as part of your preparation. **We will review answers to many of these questions in class on Friday, February 16, so please try to answer as many as you can before class.**

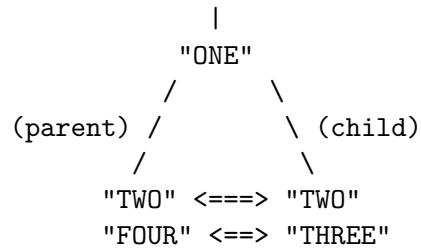
Sample Exam 1 Questions

1. Given the following C program, what is the **exact** output? If multiple outputs are possible, succinctly describe all possibilities. Assume all system calls complete successfully. Also assume that the parent process ID is 128, with any child processes numbered 768, 769, 770, etc.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    int rc;
    printf( "ONE\n" );
    rc = fork();
    printf( "TWO\n" );
    if ( rc == 0 ) { printf( "THREE\n" ); }
    if ( rc > 0 ) { printf( "FOUR\n" ); }
    return EXIT_SUCCESS;
}
```

Solution:



Here are the four possibilities.

ONE	ONE	ONE	ONE
TWO	TWO	TWO	TWO
TWO	THREE	FOUR	TWO
THREE	TWO	TWO	FOUR
FOUR	FOUR	THREE	THREE

Within the parent or the child process, the prints must be in order, but the order of prints between the processes can be intermixed in any order.

2. Given the following C program, what is the **exact** terminal output? If multiple outputs are possible, succinctly describe all possibilities. Assume that all system calls complete successfully. Further, assume that the parent process ID is 128, with any child processes numbered 768, 769, 770, etc.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int x = 150;
    printf( "PARENT: x is %d\n", x );

    printf( "PARENT: forking...\n" );
    pid_t pid = fork();
    printf( "PARENT: forked...\n" );

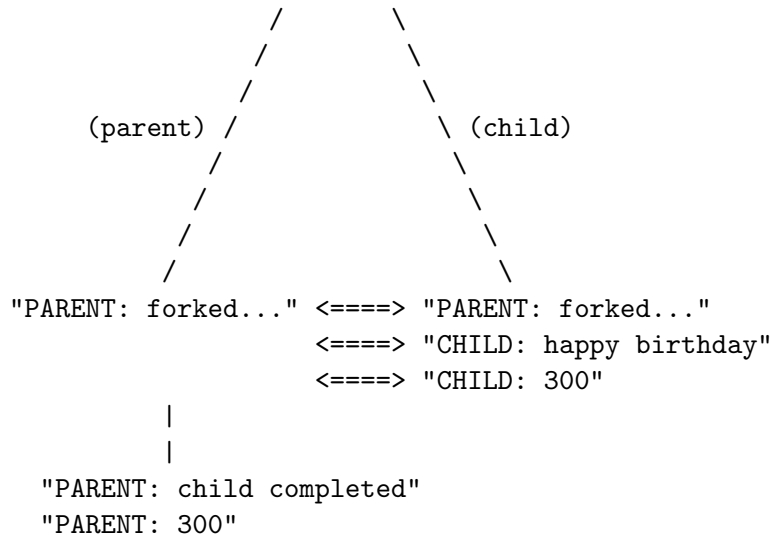
    if ( pid == 0 )
    {
        printf( "CHILD: happy birthday\n" );
        x *= 2;
        printf( "CHILD: %d\n", x );
    }
    else
    {
        waitpid( -1, NULL, 0 );

        printf( "PARENT: child completed\n" );
        x *= 2;
        printf( "PARENT: %d\n", x );
    }

    return EXIT_SUCCESS;
}
```

Solution:

```
      |
      |
      | "PARENT: x is 150"
      | "PARENT: forking...\n"
      |
      / \
     /   \
    /     \
   /       \
```



Here are the three possibilities.

```

"PARENT: x is 150"
"PARENT: forking...n"
"PARENT: forked..."
"PARENT: forked..."
"CHILD: happy birthday"
"CHILD: 300"
"PARENT: child completed"
"PARENT: 300"
  
```

```

"PARENT: x is 150"
"PARENT: forking...n"
"PARENT: forked..."
"CHILD: happy birthday"
"PARENT: forked..."
"CHILD: 300"
"PARENT: child completed"
"PARENT: 300"
  
```

```

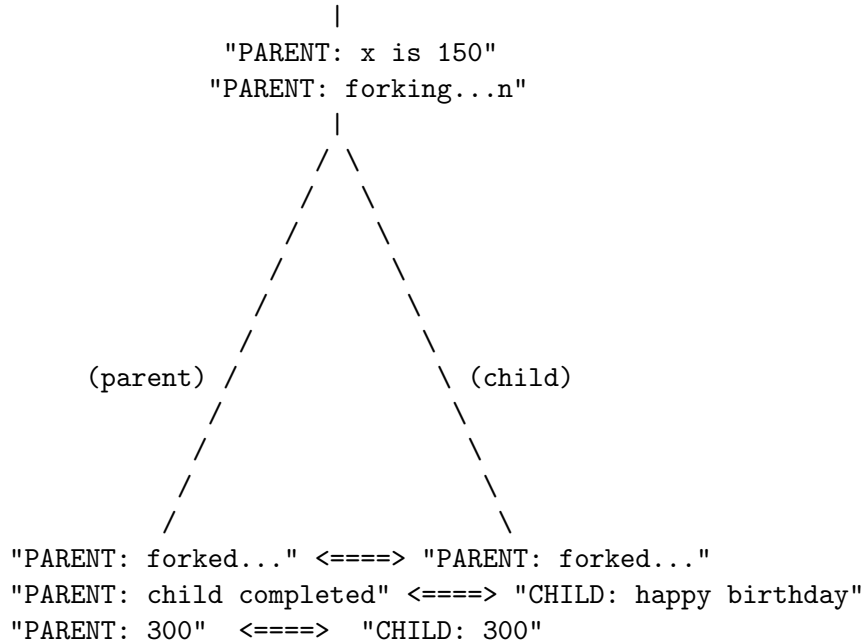
"PARENT: x is 150"
"PARENT: forking...n"
"PARENT: forked..."
"CHILD: happy birthday"
"CHILD: 300"
"PARENT: forked..."
"PARENT: child completed"
"PARENT: 300"
  
```

The two lines before the `fork()` must be printed first, and both lines after the `waitpid()` must be printed last. Within the rest, all of the `CHILD` processes must occur in order, but

the PARENT print can be interspersed anywhere within the CHILD prints.

Solution:

How would the output change if the `\verb+waitpid()+` system call was removed from the code?



The two lines before the `fork()` must be printed first. The remaining three lines on the PARENT and CHILD can be interspersed in any order so long as the prints on the PARENT and CHILD remain in the same order relative to how they were printed on the PARENT or CHILD.

You can enumerate the possibilities yourself, but we are really looking for the diagram.

How would the output change if the `waitpid()` system call was removed from the code, and the file was run as `a.out > file.txt`?

Solution:

The output would be written to the file which would change the line based buffering to block buffering. The buffer on both the PARENT and the CHILD would contain the initial two prints (originally only on the PARENT) and would be printed from each process. Additionally, all the prints from the CHILD and all the prints from the PARENT would be blocked, so there would only be two possibilities.

```
"PARENT: x is 150"
"PARENT: forking...n"
"PARENT: forked..."
"PARENT: child completed"
"PARENT: 300"
"PARENT: x is 150"
"PARENT: forking...n"
```

```
"PARENT: forked..."
"CHILD: happy birthday"
"CHILD: 300"

"PARENT: x is 150"
"PARENT: forking...n"
"PARENT: forked..."
"CHILD: happy birthday"
"CHILD: 300"
"PARENT: x is 150"
"PARENT: forking...n"
"PARENT: forked..."
"PARENT: child completed"
"PARENT: 300"
```

How would the output change if variable x started at -150 instead?

Solution:

The 150 would become -150 and the 300 would be -300. Everything else would remain the same.

3. Given the following C program, what is the **exact** terminal output? If multiple outputs are possible, succinctly describe all possibilities. Assume that all system calls complete successfully.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
    char * a = "POLYTECHNIC";
    char * b = a;
    char * c = calloc( 100, sizeof( char ) );

    printf( "[%s] [%s] [%s]\n", a + 10, b + 9, c + 8 );

    char ** d = calloc( 100, sizeof( char * ) );
    d[7] = calloc( 20, sizeof( char ) );
    d[6] = c;
    strcpy( d[7], b + 5 );
    strcpy( d[6], b + 4 );

    printf( "[%s] [%s] [%s]\n", d[7], d[6], c + 5 );

    float e = 2.71828;
    float * f = calloc( 1, sizeof( float ) );
    float * g = f;
    float * h = &e;

    printf( "[%3.2f] [%2.2f] [%2.1f]\n", *f, *g, *h );

    return EXIT_SUCCESS;
}
```

Add code to deallocate all dynamically allocated memory.

Remove all of the square brackets, '[' and ']' from the code above (ignore the brackets in the print statements).

Solution:

Run it yourself and work it out! Use valgrind.

How does the output change if `malloc()` is used instead of `calloc()`?

Solution:

Run it yourself and see. The short answer is that you cannot guarantee that memory is 0 unless it has been specifically set. This may affect primarily the first and last prints where `c` has not been initialized in the first print and `f` and `g` have not been initialized. The first print is possibly the most important since a `\0` is necessary to terminate a string and the

print statement may go beyond the allocated memory. (It would be more problematic if we were copying from `c` rather than into `c` later in the program.)

4. Given the following C program, what is the **exact** terminal output? If multiple outputs are possible, succinctly describe all possibilities. Assume that all system calls complete successfully.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    printf( "ONE\n" );
    fprintf( stderr, "ERROR: ONE\n" );
    int rc = close( 2 );
    printf( "==> %d\n", rc );

    printf( "TWO\n" );
    fprintf( stderr, "ERROR: TWO\n" );
    rc = dup2( 1, 2 );
    printf( "==> %d\n", rc );

    printf( "THREE\n" );
    fprintf( stderr, "ERROR: THREE\n" );

    return EXIT_SUCCESS;
}
```

Solution:

Run it yourself and work it out!

5. Given the following C program, what is the **exact** terminal output? Further, what is the **exact** contents of the `output.txt` file? If multiple outputs are possible, succinctly describe all possibilities. Assume that all system calls complete successfully.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    int fd;
    close( 2 );
    printf( "HI\n" );

    #if 0
        close( 1 ); /* <== add this line later.... */
    #endif

    fd = open( "output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600 );

    printf( "==> %d\n", fd );
    printf( "WHAT?\n" );
    fprintf( stderr, "ERROR\n" );

    close( fd );

    return EXIT_SUCCESS;
}
```

Solution:

Run it yourself and work it out!

How do the output and file contents change if the second `close()` system call is uncommented?

Solution:

Run it yourself and work it out!

Add code to redirect all output on `stdout` and `stderr` to the output file.

Solution:

`\textbf{Solution:}`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
```

```

int main()
{
    int fd;

    fd = open( "output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600 );
    dup2(fd, 1);
    dup2(fd, 2);
    printf( "HI\n" );

    #if 0
        close( 1 ); /* <== add this line later.... */
    #endif

    fd = open( "output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600 );

    printf( "==> %d\n", fd );
    printf( "WHAT?\n" );
    fprintf( stderr, "ERROR\n" );

    close( fd );

    return EXIT_SUCCESS;
}

```

6. Given the following C program, what is the **exact** output? Further, what is the **exact** contents of the `output.txt` file? If multiple outputs are possible, succinctly describe all possibilities. Assume that all system calls complete successfully. Further, assume that the parent process ID is 128, with any child processes numbered 768, 769, 770, etc.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int fd;
    close( 2 );
    printf( "HI\n" );

    #if 0
        close( 1 ); /* <== add this line later.... */
    #endif

    fd = open( "output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600 );

    printf( "==> %d\n", fd );
    printf( "WHAT?\n" );
    fprintf( stderr, "ERROR\n" );

    int rc = fork();

    if ( rc == 0 )
    {
        printf( "AGAIN?\n" );
        fprintf( stderr, "ERROR ERROR\n" );
    }
    else
    {
        waitpid( -1, NULL, 0 );
    }

    printf( "BYE\n" );
    fprintf( stderr, "HELLO\n" );
    close( fd );
    return EXIT_SUCCESS;
}
```

How do the output and file contents change if the second `close()` system call is uncommented?

Solution:

Run the code to find out.

7. Given the following C program, what is the **exact** output? If multiple outputs are possible, succinctly describe all possibilities. Assume that all system calls complete successfully. Further, assume that the parent process ID is 128, with any child processes numbered 768, 769, 770, etc.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    int rc;
    int p[2];
    rc = pipe( p );
    printf( "%d %d %d\n", getpid(), p[0], p[1] );

    rc = fork();

    if ( rc == 0 )
    {
        rc = write( p[1], "ABCDEFGHJKLMNOPQRSTUVWXYZ", 26 );
    }

    if ( rc > 0 )
    {
        char buffer[70];
        rc = read( p[0], buffer, 8 );
        buffer[rc] = '\0';
        printf( "%d %s\n", getpid(), buffer );
    }

    printf( "BYE\n" );

    return EXIT_SUCCESS;
}
```

Solution:

This is a sneaky one.

```

      128 3 4
      |
      |
      fork()
      /  \
  (parent)/  \ (child)
      /        \
128 IJKLMNOP or 128 ABCDEFGH <====> 768 ABCDEFGH or 768 IJKLMNOP
```

BYE <====> BYE

128 3 4	128 3 4	128 3 4	128 3 4
128 ABCDEFGH	128 ABCDEFGH	768 IJKLMNOP	768 IJKLMNOP
BYE	768 IJKLMNOP	128 ABCDEFGH	BYE
768 IJKLMNOP	BYE	BYE	128 ABCDEFGH
BYE	BYE	BYE	BYE
128 3 4	128 3 4	128 3 4	128 3 4
768 ABCDEFGH	768 ABCDEFGH	128 IJKLMNOP	128 IJKLMNOP
BYE	128 IJKLMNOP	768 ABCDEFGH	BYE
128 IJKLMNOP	BYE	BYE	768 ABCDEFGH
BYE	BYE	BYE	BYE

8. Given the table of process burst times, arrival times, and priorities shown below, calculate the turnaround time, wait time, and number of preemptions each process experiences using the following scheduling algorithms: FCFS; SJF; SRT; RR with a time slice of 3 ms; and Priority scheduling with FCFS as secondary scheduling algorithm (note that the lower the priority value, the higher the priority).

PROCESS	BURST TIME	ARRIVAL TIME	PRIORITY
P1	7 ms	0 ms	2
P2	5 ms	0 ms	3
P3	5 ms	1 ms	1
P4	6 ms	4 ms	2

Solution:

Here is the SRT algorithm.

```
=====
P1 >                xxxxxxxx
P2 >xxxxxx
P3 |>   xxxxxx
P4 |  >    xxxxxx
=====
                1111111112222
            12345678901234567890123
```

P1 has a Turnaround time of $23-0 = 23\text{ms}$ and a wait time of $23-7 = 16\text{ms}$
P2 has a Turnaround time of $5-0 = 5\text{ms}$ and a wait time of $5-5 = 0\text{ms}$
P3 has a Turnaround time of $10-1 = 9\text{ms}$ and a wait time of $9-5 = 4\text{ms}$
P4 has a Turnaround time of $16-4 = 12\text{ms}$ and a wait time of $12-6 = 6\text{ms}$

Here is the Priority algorithm. You can work through the rest on your own.

```
=====
P1 >x   xxxxxx
P2 >                xxxxxx
P3 |>xxxxxx
P4 |  >    xxxxxx
=====
                1111111112222
            12345678901234567890123
```

P1 has a Turnaround time of $12-0 = 12\text{ms}$ and a wait time of $12-7 = 5\text{ms}$
P2 has a Turnaround time of $23-0 = 23\text{ms}$ and a wait time of $23-5 = 18\text{ms}$
P3 has a Turnaround time of $6-1 = 5\text{ms}$ and a wait time of $5-5 = 0\text{ms}$
P4 has a Turnaround time of $18-4 = 14\text{ms}$ and a wait time of $14-6 = 8\text{ms}$

9. Write pseudocode for the scheduling algorithms above; for each, determine the computational complexity of adding a process to the ready queue, of selecting the next process, and of

removing a process from the queue. (Note that this is a practice question and is too general to appear on the exam.)

Solution:

This is kind of a conceptual question, but here is what I would expect for SRT (Shortest Remaining Time).

```
insert_process_in_queue(new_process, running_process, queue)
  if there is no running process
    Move the new process to the CPU
  else if the new_process has a shorter
    expected burst time than the running process
    Move the new process to the CPU
    Place the old process at the front of the queue
  else
    Do a binary search of the queue to place the new_process correctly
end #  $O(\log(N))$  in the size of the READY QUEUE

select_next_process(queue)
  Take the first process in the queue, call it P
  remove_process_from_queue(P, queue)
  return P
end #  $O(1)$ 

remove_process_from_queue(p, queue)
  set the queue head to the next process in the queue
end #  $O(1)$ 

main()
  make a ready_queue with pointer head
  while TRUE
    wait for event
    if the event is:
      process P entered into the system:
        insert_process_in_queue(P, ready_queue)
      process P moved to ready state:
        insert_process_in_queue(P, ready_queue)
      current running process terminated burst:
        calculate new time estimate for running process
        move running process to I/O subsystem, or remove if terminated`
      P = select_next_process(ready_queue)
      move P to the CPU if it exists.
    end while
  end main
```

And here is what I would expect for RR (Round Robin).

```

insert_process_in_queue(new_process, running_process, queue)
    if there is no running process
        Move new_process to the CPU
    else
        Place new_process at the end of the queue
end # O(1)

select_next_process(queue)
    Take the first process in the queue, call it P
    remove_process_from_queue(P, queue)
    return P
end # O(1)

remove_process_from_queue(p, queue)
    set the queue head to the next process in the queue
end # O(1)

main()
    make a ready_queue with pointer head
    while TRUE
        wait for event
        if the event is:
            process P entered into the system:
                insert_process_in_queue(P, ready_queue)
            process P moved to ready state:
                insert_process_in_queue(P, ready_queue)
            current running process terminated burst:
                move running process to I/O subsystem, or remove if terminated`
                P = select_next_process(ready_queue),
                move P to the CPU if it exists.
            current running process used up its time slice and ready_queue is not empty:
                insert_process_in_queue(current running process, ready_queue)
                P = select_next_process(ready_queue),
                move P to the CPU
            current running process used up its time slice and ready_queue is empty:
                Reset the time slice
        end while
    end main

```

The rest are left for you. Basically, think of what kind of structures you need. SRT, Priority, and SJF all need a priority queue which implies ordering. This can be maintained in $O(\log(N))$ time. All deletions are from the front, which can be maintained in constant time.

RR and FCFS do not need a priority queue and just need removal from the front and insertion at the end. Both of which can be implemented in constant time.

10. For each scheduling algorithm above, describe whether CPU-bound or I/O-bound processes are favored. Remember that a CPU-bound process is a process with long CPU burst times,

whereas an I/O-bound process is a process with short CPU burst times and long I/O bursts.

Solution:

I/O bound processes benefit from the SJT and SRT algorithms. CPU bound processes benefit from FCFS since they are not penalized for their long running times and do not need to give up the CPU once they have it. Priority benefits higher priority processes and depends on how the priority is set. RR is neutral. Every process gets an equal share of the processor.

11. For each scheduling algorithm above, describe how starvation can occur (or describe why it never will occur). What about indefinite blocking?

Solution:

Starvation or indefinite blocking can occur in SJF, SRT, and in Priority without aging. They allow higher priority (or shorter) jobs to jump ahead of the READY queue. It can also occur in RR if new jobs are allowed to jump to the front of the queue.

FCFS, Round Robin, and priority with aging ensure that every process gets a shot at the front of the READY Queue. RR and FCFS only add processes at the end of the queue, so your position in the queue is fixed at the time you enter and you are guaranteed to get to the front. Priority with aging ensures that your process gains priority as it waits and it must be the highest priority process at some point.

12. Given actual burst times for process P shown below and an initial guess of 6 ms, calculate the estimated next burst time for P by using exponential averaging with alpha (α) set to 0.5; recalculate with α set to 0.25 and 0.75.

Measured burst times for P are: 12 ms; 8 ms; 8 ms; 9 ms; and 4 ms.

Solution:

For alpha = 0.5:

$$\begin{aligned}\tau_0 &= 6ms \\ \tau_1 &= 0.5 * 12 + 0.5 * 6 = 9ms \\ \tau_2 &= 0.5 * 8 + 0.5 * 9 = 8.5ms \\ \tau_3 &= 0.5 * 8 + 0.5 * 8.5 = 8.25ms \\ \tau_4 &= 0.5 * 9 + 0.5 * 8.25 = 8.625ms \\ \tau_5 &= 0.5 * 4 + 0.5 * 8.625 = 6.3125ms\end{aligned}$$

For alpha = 0.25:

$$\begin{aligned}\tau_0 &= 6ms \\ \tau_1 &= 0.25 * 12 + 0.75 * 6 = 7.5ms \\ &\dots\end{aligned}$$

From here it is just math.

13. Describe what happens when a child process terminates; also, what happens when a process that has child processes terminates?

Solution:

- (a) When a child process terminates, it becomes a Zombie until it can return its final status to its parent
- (b) When a parent process terminates, any children who haven't reorted status are inherited by Process 1.

14. In a shell, how can pipe functionality be implemented? Describe in detail.

Solution:

- (a) Parent creates a pipe()
- (b) Parent forks the first child (left command)
 - i. child closes read end of pipe
 - ii. child uses dup2 to overwrite write end of pipe onto stdout
 - iii. child executes command
- (c) Parent forks the second child (right command)
 - i. child closes write end of pipe
 - ii. child uses dup2 to overwrite read end of pipe onto stdin
 - iii. child executes command
- (d) parent closes both ends of pipe

15. Why might `fork()` fail? Why does a “fork-bomb” cause a system to potentially crash and how can a “fork-bomb” be avoided by an operating system?

Solution:

- (a) It can potentially create infinite processes which consume all system resources.
- (b) By setting limits on forking such as shown by `rlimit()`

16. Why is it important to use `free()` to deallocate dynamically allocated memory?

Solution:

- (a) To avoid memory leaks particularly in long running programs.
- (b) To get good grades on your homeworks.

17. Describe (using code snippets) three ways to cause a segmentation fault.

Solution: Google “what are three ways to cause a segmentation fault”

18. Describe (using code snippets) three ways to cause a memory leak.

Solution: Google “what are three ways to cause a memory leak”

19. Describe (using code snippets) three ways to cause a buffer overflow.

Solution: Google “what are three ways to cause a buffer overflow”

20. Why is output to `stdout` and other file descriptors buffered by default?

Solution: For program efficiency. It allows the program to continue to execute without having to invoke the kernel to execute I/O.